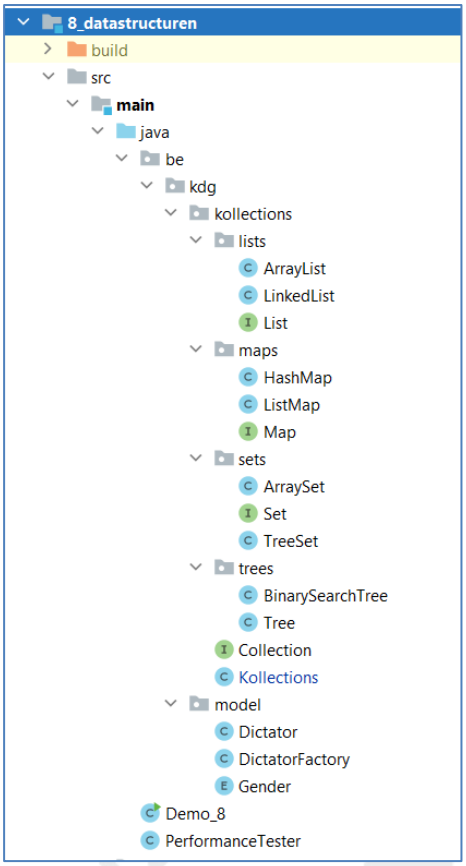
GROEIPROJECT
MODULE 8_BIS: DATASTRUCTUREN EN ALGORITMES (DEEL II)

We maken geen nieuwe module deze week; we werken dus verder in de achtste module: `m8_datastructuren`



5. Map



5.1. Refactor de module m8_datastructuren zodat je onder de package kollections subpackages maakt: lists, maps, sets en trees. De uiteindelijke packagestructuur ziet er dan zo uit →

5.2. Kopieer de Collection interface uit het voorbeeldproject.

5.3. Aanpassingen aan de `List` interface: overerven van Collection en de volgende methode toevoegen:
- `int indexOf(E element)`:

5.4. Pas nu de List-implementaties `ArrayList` en `LinkedList` aan en werk alle ontbrekende methoden correct uit. Maak in de `indexOf`-methode gebruik van `Kollections.linearSearch`

5.5. Kopieer vanuit het voorbeeldproject de set- en map-interfaces en hun implementaties naar jouw GP.
Opgelet dat je de juiste imports doet! Dus niet per ongeluk de echte java.util.ArrayList importen!

5.6. In de klasse `HashMap` werk je de methode `put` en `get` uit.

5.7. Pas in de basisklasse de methoden `equals` en `hashCode` aan, zodat ze testen op één identificerend String-attribuut (in ons voorbeeld: de **naam** van de Dictator). Zorg er ook voor dat de `compareTo` in de basisklasse op hetzelfde String-attribuut vergelijkt.

5.8. In de klasse `PerformanceTester` schrijf je een nieuwe methode `compareListMapToHasMap`:
- Als parameter komt de grootte van de map binnen.
- Maak een `ListMap` en `HashMap` aan (Maak liefst gebruik van een aparte private methode `fillMap` die dat doet). Maak basisobjecten aan om de map te vullen (via de factory). Het identificerend attribuut (zie 5.7) van je basisobject is een vaste string met als suffix een teller. In ons voorbeeld is dat de naam van de dictators: "Pol1", "Pol2",... Gebruik het basisobject als **Key** en geef een tekst mee als **Value**. In ons voorbeeld is dat "Ik ben de waarde Pol1"...
- Zorg ervoor dat de waarde die je gebruikt als sleutel een vast woord is met een teller als suffix, zodat je de basisobjecten straks gemakkelijk kan opzoeken. In ons voorbeeld heten onze dictators Pol1, Pol2,...
- Voorzie in de basisklasse een public static `equalsCounter` die telkens verhoogd wordt als de `equals`-methode wordt uitgevoerd.

- Doe 1000 keer een get in beide maps, telkens van een andere dictator (Pol1, Pol2…). Meet de benodigde tijd (System.nanoTime) en het aantal vergelijkingen (`equalsCounter`)

  Mogelijke output:

```
Listmap: n = 1000, equalscount =   500500, nanosec =   30235000
Hashmap: n = 1000, equalscount =     5599, nanosec =    2419300
```



6. Tree

6.1. Kopieer vanuit het voorbeeldproject de `Tree`-implementaties naar jouw GP.

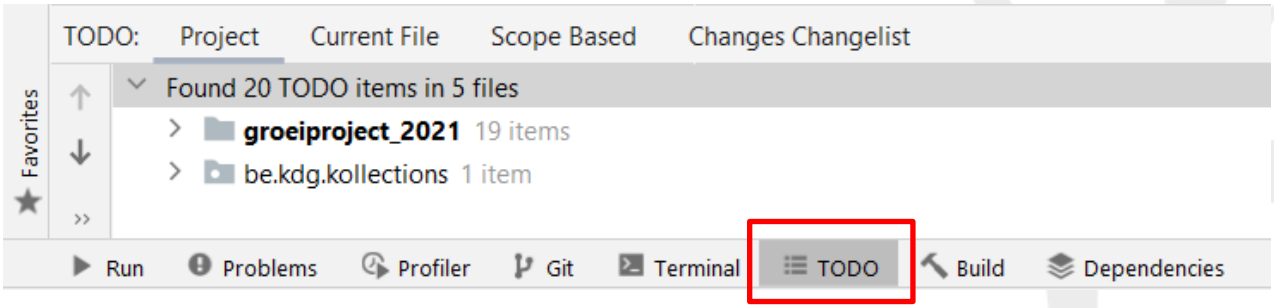
6.2. Werk de klasse `TreeSet` uit: die werkt identiek zoals een `BinarySearchTree`, alleen zijn alle elementen uniek.
- Implementeer daar zeker de `Set`-interface!
- Pas de `add`-methode aan zodat er geen duplicaten worden toegevoegd.

6.3. In de klasse `PerformanceTester` schrijf je een nieuwe methode `compareArraySetToTreeSet`:
- Gebruik de factory om 1000 random basisobjecten in de map toe te voegen.
- Meet de benodigde stappen (zowel `equalsCounter` als `compareCounter`) en verstreken tijd (System.nanoTime)

  Mogelijke output (kan je de resultaten verklaren?):

```
ArraySet, n = 1000: equalscount: 499500
ArraySet, n = 1000: comparecount: 0
ArraySet, n = 1000: nanosec: 16478700
TreeSet,  n = 1000: equalscount: 0
TreeSet,  n = 1000: comparecount: 15918
TreeSet,  n = 1000: nanosec: 3578100
```

Controleer onderaan in je TODO-venster of er geen openstaande TODO's meer zijn:



Controleer of alles werkt en pushen je code naar je repo op GitLab…